



Ritoto Express

Architecture du code & Guide de déploiement

Django 4.2

React 18 + Vite

PostgreSQL 16

Redis

Django Channels

Docker

Nginx + SSL

 ritoto-campus.com → Frontend React

 api.ritoto-campus.com → API Django + WebSockets

VPS Contabo · Mars 2026 · Version 2.0

Table des matières

Partie 1 — Architecture du code

- 1.1 Vue d'ensemble de la stack
- 1.2 Structure du projet Django (backend)
- 1.3 Applications Django
- 1.4 Configuration des settings (dev / prod)
- 1.5 Système WebSocket temps réel
- 1.6 Structure du frontend React
- 1.7 Flux des données — de la commande à la notification
- 1.8 Système de frais de service

Partie 2 — Déploiement sur VPS Contabo

- 2.1 Architecture cible en production
- 2.2 Prérequis VPS & DNS
- Phase 1 — Préparer le serveur
- Phase 2 — PostgreSQL & migration
- Phase 3 — Configuration production
- Phase 4 — Docker en production
- Phase 5 — Nginx & SSL Let's Encrypt
- Phase 6 — Cache Redis
- Phase 7 — Système de backup
- Phase 8 — Mise en ligne & checklist
- Phase 9 — Opérations courantes

Annexes

- Variables d'environnement complètes
- Points d'attention & sécurité

Stack technique, organisation du projet, flux de données et WebSockets

1.1 Vue d'ensemble de la stack

Ritoto Express est une application de commande de repas pour campus universitaire. Elle repose sur une architecture découplée : un backend API REST Django et un frontend React, communiquant via HTTP pour les opérations classiques et via WebSocket pour les notifications temps réel.

1.2 Structure du projet Django

1.3 Applications Django

● authentication

- Modèle `Utilisateur` (`AbstractBaseUser`)
- Rôles : ETUDIANT, ADMIN, CHEF_SECTEUR, LIVREUR
- Authentification par email + mot de passe
- Token DRF pour les requêtes API
- Inscription avec code étudiant

● administration

- Modèles : `Produit` , `Secteur` , `Salle`
- Modèle `Configuration` (taux frais de service, heures ouverture)
- Vue comptabilité (CA, frais, par secteur)
- Admin Django avec `simpleui`

● commandes

- Modèle `Commande` + `LigneCommande`
- Statuts : EN_ATTENTE → VALIDEE → PRETE → DISTRIBUEE
- Intégration paiement Senfenico (webhook)
- Consumer WebSocket `CommandeConsumer`
- Notifications push automatiques

● config/asgi.py

- Routeur ASGI : HTTP → Django, WS → Channels
- `AllowedHostsOriginValidator` (sécurité WS)
- `AuthMiddlewareStack` (authentification WS)
- URL WebSocket : `/ws/commandes/`

1.4 Configuration des settings (dev / prod)

Les settings sont organisés en 3 fichiers pour séparer clairement le développement de la production :

base.py

- INSTALLED_APPS
- MIDDLEWARE
- REST_FRAMEWORK
- AUTH_USER_MODEL
- Email (SMTP)
- Senfénico keys
- ASGI_APPLICATION
- Internationalisation

development.py

- DEBUG = True
- SECRET_KEY dev
- ALLOWED_HOSTS = ['*']
- PostgreSQL local
- InMemoryChannelLayer
- Cache local mémoire
- CORS : tout autorisé
- Email console

production.py

- DEBUG = False
- SECRET_KEY = .env
- ALLOWED_HOSTS strict
- PostgreSQL (docker)
- RedisChannelLayer
- Redis cache + sessions
- CORS origin list
- HTTPS headers sécurisés



Activation : La variable d'environnement `DJANGO_SETTINGS_MODULE` détermine quel fichier est chargé. En développement : `config.settings.development` . En production : `config.settings.production` (défaut dans `wsgi.py`).

1.5 Système WebSocket temps réel

Django Channels étend Django pour gérer les connexions persistantes (WebSocket). Redis sert de canal de communication entre les processus.

Groupes et destinataires

Groupe WebSocket	Destinataire	Événements reçus
<code>user_{id}</code>	Étudiant	Mises à jour du statut de ses commandes, confirmation paiement
<code>chef_{secteur_id}</code>	Chef de secteur	Nouvelle commande reçue dans son secteur
<code>admin</code>	Administrateur	Toutes les nouvelles commandes, alertes générales

Flux d'une notification

```
1. Étudiant passe une commande → POST /api/commandes/ → Créé en base, statut = EN_ATTENTE → notifier_nouvelle_commande(commande) → channel_layer.group_send("chef_{id}", {...}) → channel_layer.group_send("admin", {...}) → Chef et admin reçoivent la notif en temps réel 2. Chef valide la commande →
```

```
POST /api/commandes/{id}/valider/ → statut → VALIDEE, save() →  
envoyer_mise_a_jour_commande(commande) →  
channel_layer.group_send("user_{etudiant_id}", {...}) → L'étudiant voit "Commande  
VALIDEE ✓" instantanément → Toast flash + badge notification dans le header
```

Composants frontend

Fichier	Rôle
<code>hooks/useWebSocket.js</code>	Ouvre la connexion WS, reconnexion automatique (backoff exponentiel : 1s, 2s, 4s... max 30s)
<code>stores/notificationsStore.js</code>	Store Zustand : liste des notifications, compteur non-lues, marquer comme lu
<code>components/NotificationBell.jsx</code>	Cloche dans le topbar, badge rouge, dropdown avec liste des notifications
<code>layouts/DashboardLayout.jsx</code>	Monte le hook WS dès la connexion, appelle <code>addNotification()</code> + toast à chaque message

1.6 Structure du frontend React

```
frontend/ ├── src/ | ├── pages/ | | ├── auth/ ← Connexion, inscription | | ├── student/ ← Dashboard, Produits, Panier, Commandes, Plaintes, Profil | | ├── admin/ ← Dashboard, Utilisateurs, Produits, Commandes, Comptabilité, Config | | ├── chef/ ← Dashboard, Commandes secteur | | ├── delivery/ ← Dashboard livreur | ├── layouts/ | | ├── DashboardLayout.jsx ← Sidebar + topbar + WS hook + NotificationBell | ├── components/ | | ├── NotificationBell.jsx ← Cloche + dropdown notifications | | ├── Badge.jsx ← Badge statut commande coloré | | ├── ... | ├── stores/ | | ├── authStore.js ← Zustand : user, token, login/logout | | ├── cartStore.js ← Zustand : panier (items, add, remove) | | ├── notificationsStore.js ← Zustand : notifications temps réel | ├── hooks/ | | ├── useWebSocket.js ← Hook WS avec reconnexion auto | ├── api/ ← Fonctions Axios vers l'API | ├── utils/ | ├── receipt.js ← Génération reçu HTML → impression | ├── vite.config.js ← Proxy /api et /ws en dev, build versionné | └── index.html
```

1.7 Flux des données — de la commande à la notification

Étape	Acteur	Action	Notification envoyée
1	Étudiant	Place une commande (panier → validation)	Chef + Admin : "Nouvelle commande de {nom}"
2	Étudiant	Paye via Senfenico (webhook reçu)	Étudiant : "Paieement reçu ! Commande en préparation"
3	Chef secteur	Valide la commande	Étudiant : "Votre commande est validée ✓"
4	Chef secteur	Marque la commande comme prête	Étudiant : "Votre commande est prête !"
5	Livreur	Distribue la commande	Étudiant : "Commande livrée. Bon appétit ! 🍴"
—	Chef secteur	Rejette la commande	Étudiant : "Commande rejetée : {motif} ❌"

1.8 Système de frais de service

Les frais sont unifiés en un seul taux configurable. Il n'y a plus de frais de livraison séparés, ni de frais par méthode de paiement.



Calcul des frais

```
frais_service = total_ht × taux_service / 100
total_ttc = total_ht + frais_service

# Exemple : total_ht = 3 000 FCFA, taux = 10 %
# frais_service = 300 FCFA
# total_ttc = 3 300 FCFA
```

Le taux est configurable depuis l'interface d'administration (modèle `Configuration.taux_service`, défaut 10%). Le client voit uniquement "Frais de service (10%)" — sans détail de la composition.

Déploiement sur VPS Contabo

Roadmap complète en 9 phases — du serveur vierge à la mise en production

2.1 Architecture cible en production

```
Internet | ▼ [ Nginx ] — SSL Let's Encrypt (Certbot) | — ritoto-campus.com | —  
Frontend React (Nginx → dist/ statique) | — api.ritoto-campus.com — /api/* →  
backend:8000 (Gunicorn — requêtes HTTP REST) — /ws/* → daphne:8001 (Django Channels  
— WebSockets) — /static/* → volume static_data — /media/* → volume media_data  
Conteneurs Docker : backend | Gunicorn, port 8000 daphne | Daphne ASGI, port 8001  
frontend | Nginx (build Vite), port 80 nginx | Reverse proxy, ports 80 + 443 redis |  
Cache + sessions + channel layer WS postgres | Base de données principale
```

Volumes persistants

Volume	Contenu	Criticité
postgres_data	Données PostgreSQL	● Critique
redis_data	Persistance Redis	● Moyenne
media_data	Images produits, reçus	● Haute
static_data	Fichiers statiques Django	● Moyenne
certs	Certificats SSL Let's Encrypt	● Haute

2.2 Prérequis VPS & DNS



Spécifications recommandées

OS Ubuntu 22.04 LTS

RAM 4 Go minimum — 8 Go recommandé

Stockage	40 Go SSD minimum
----------	-------------------

CPU	2 vCPU minimum
-----	----------------

Logiciels	Docker Engine, Docker Compose v2, Git, Certbot
-----------	--

Configuration DNS (chez le registrar)

```
ritoto-campus.com      A → <IP_VPS>
www.ritoto-campus.com  A → <IP_VPS>
api.ritoto-campus.com  A → <IP_VPS>
```



Propagation DNS : Les enregistrements DNS peuvent prendre jusqu'à 24–48h pour se propager. Obtenir le SSL avant que les DNS ne pointent vers le VPS est impossible. Configurer les DNS **avant** d'aller plus loin.

PHASE 1

Préparer le serveur

- ☐ Louer et initialiser le VPS Contabo (Ubuntu 22.04)
- ☐ Créer un utilisateur non-root avec accès sudo (ne jamais travailler en root)
- ☐ Configurer le pare-feu UFW : ouvrir ports 80, 443, 22 (SSH)
- ☐ Installer Docker Engine (via le script officiel docs.docker.com)
- ☐ Installer Docker Compose v2 (`docker compose version` pour vérifier)
- ☐ Installer Certbot : `apt install certbot python3-certbot-nginx`
- ☐ Pointer les enregistrements DNS vers l'IP du VPS
- ☐ Cloner le dépôt Git sur le VPS : `/opt/ritoto-express/`

PHASE 2

PostgreSQL & migration des données

- ☐ S'assurer que `psycopg2-binary` est dans `requirements.txt`
- ☐ Créer le fichier `.env` sur le VPS à partir du modèle `.env.example`
- ☐ Définir `POSTGRES_DB` , `POSTGRES_USER` , `POSTGRES_PASSWORD`
- ☐ Si migration depuis un autre environnement : exporter avec `python manage.py dumpdata > data.json`
- ☐ Démarrer PostgreSQL seul : `docker compose -f docker-compose.prod.yml up -d postgres`
- ☐ Appliquer les migrations : `docker compose exec backend python manage.py migrate`
- ☐ Importer les données : `docker compose exec backend python manage.py loaddata data.json`
- ☐ Vérifier l'intégrité des données dans Django admin

PHASE 3

Configuration production — variables d'environnement

- ☐ Générer `DJANGO_SECRET_KEY` forte et unique (ex: `python -c "import secrets; print(secrets.token_hex(50))"`)
- ☐ Définir `DEBUG=False` et `DJANGO_SETTINGS_MODULE=config.settings.production`
- ☐ Configurer `DJANGO_ALLOWED_HOSTS=api.ritoto-campus.com`
- ☐ Configurer `CORS_ALLOWED_ORIGINS=https://ritoto-campus.com`
- ☐ Vérifier les clés Senfénico **production** (distinctes des clés de test)
- ☐ Configurer les identifiants email SMTP production
- ☐ Définir `FRONTEND_URL=https://ritoto-campus.com`

Variables d'environnement production (.env)

```
# Django
DJANGO_SECRET_KEY=<clé longue 100+ chars>
DJANGO_SETTINGS_MODULE=config.settings.production
DJANGO_ALLOWED_HOSTS=api.ritoto-campus.com
FRONTEND_URL=https://ritoto-campus.com
CORS_ALLOWED_ORIGINS=https://ritoto-campus.com
CSRF_TRUSTED_ORIGINS=https://api.ritoto-campus.com

# PostgreSQL
POSTGRES_DB=ritoto
POSTGRES_USER=ritoto
POSTGRES_PASSWORD=<mot de passe fort>
POSTGRES_HOST=postgres

# Redis
REDIS_URL=redis://redis:6379/0

# Paiement Senfenico (clés production)
SENFENICO_API_KEY=<clé prod>
SENFENICO_WEBHOOK_SECRET=<secret prod>

# Email
EMAIL_HOST=smtp.gmail.com
EMAIL_HOST_USER=<adresse email>
EMAIL_HOST_PASSWORD=<mot de passe app Gmail>
DEFAULT_FROM_EMAIL=noreply@ritoto-campus.com
```

PHASE 4

Docker en production

- ☐ Vérifier que `Dockerfile.prod` est présent (expose ports 8000 et 8001)
- ☐ Vérifier que `docker-compose.prod.yml` contient tous les services
- ☐ Builder les images : `docker compose -f docker-compose.prod.yml build`
- ☐ Démarrer la stack complète : `docker compose -f docker-compose.prod.yml up -d`
- ☐ Vérifier que tous les conteneurs sont UP : `docker compose ps`
- ☐ Lancer les migrations Django : `docker compose exec backend python manage.py migrate`
- ☐ Collecter les fichiers statiques : `docker compose exec backend python manage.py collectstatic --noinput`
- ☐ Créer le superutilisateur : `docker compose exec backend python manage.py createsuperuser`

Services Docker en production

Service	Image	Rôle	Port interne
<code>backend</code>	Dockerfile.prod	Django + Gunicorn (HTTP REST)	8000
<code>daphne</code>	Dockerfile.prod	Django Channels (WebSockets)	8001
<code>frontend</code>	Nginx + build Vite	Fichiers statiques React	80
<code>nginx</code>	Nginx Alpine	Reverse proxy + SSL	80 / 443
<code>redis</code>	Redis Alpine	Cache + sessions + WS channel layer	6379
<code>postgres</code>	PostgreSQL 16	Base de données principale	5432

PHASE 5

Nginx & SSL Let's Encrypt

- ☐ Obtenir les certificats SSL AVANT de démarrer Nginx en HTTPS :

```
certbot certonly --standalone \
-d ritoto-campus.com \
-d www.ritoto-campus.com \
-d api.ritoto-campus.com
```

- ☐ Monter `/etc/letsencrypt/` en volume dans le conteneur Nginx
- ☐ Configurer le renouvellement automatique (cron) :

```
0 3 * * * certbot renew --quiet && docker compose -f docker-compose.prod.yml restart
```

☐ Vérifier que le certificat est valide : `certbot certificates`

Configuration Nginx — Routing des requêtes

<code>/ws/</code>	→ <code>daphne:8001</code>	WebSockets (upgrade HTTP → WS), proxy_read_timeout 86400
<code>/api/</code>	→ <code>backend:8000</code>	API REST Django (Gunicorn)
<code>/admin/</code>	→ <code>backend:8000</code>	Interface d'administration Django
<code>/static/</code>	→ <code>volume static_data</code>	Fichiers statiques Django (CSS, JS admin)
<code>/media/</code>	→ <code>volume media_data</code>	Fichiers media (images produits, reçus)
<code>/</code>	→ <code>frontend:80</code>	Application React (SPA, index.html)

Configuration WebSocket dans Nginx

```
location /ws/ {
    proxy_pass http://daphne:8001;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
    proxy_read_timeout 86400; # maintenir la connexion ouverte 24h
}
```

PHASE 6

Cache Redis

- ☐ Redis est automatiquement lancé via docker-compose.prod.yml
- ☐ Vérifier la connexion : `docker compose exec redis redis-cli ping` → PONG
- ☐ Cache Django configuré sur `redis://redis:6379/0`
- ☐ Sessions Django stockées dans Redis (pas en base de données)
- ☐ Channel layer WebSocket sur `redis://redis:6379/1` (DB distincte)

Stratégie de cache par type de ressource

Ressource	Cache Nginx	Cache Redis Django	Invalidation
<code>dist/assets/*.js/.css</code>	1 an (immutable)	—	Nom de fichier versionné
<code>dist/index.html</code>	no-cache	—	À chaque déploiement
<code>GET /api/produits/</code>	—	5 minutes	À la modification produit
<code>GET /api/configuration/</code>	—	10 minutes	À la sauvegarde config
<code>GET /api/commandes/</code>	no-store	—	Toujours frais
<code>/media/*</code>	7 jours	—	Rarement modifié
<code>/admin/</code>	no-store	—	Jamais caché

PHASE 7

Système de backup automatique

- ☐ Créer le script `docker/scripts/backup.sh`
- ☐ Configurer le cron pour une exécution quotidienne à 2h du matin
- ☐ Vérifier que les 14 derniers backups sont conservés (rotation automatique)
- ☐ Optionnel : configurer l'upload vers Contabo Object Storage ou Backblaze B2
- ☐ Tester une restauration complète avant la mise en production !

Ce que le backup doit couvrir

Donnée	Emplacement	Criticité	Fréquence
Base PostgreSQL	volume <code>postgres_data</code>	 Critique	Quotidien
Fichiers media	volume <code>media_data</code>	 Haute	Quotidien
Fichier <code>.env</code>	<code>/opt/ritoto-express/</code>	 Critique	À chaque modif
Certificats SSL	<code>/etc/letsencrypt/</code>	 Haute	À chaque renouvellement
Config Nginx	<code>docker/nginx/</code>	 Moyenne	Via Git

Script backup — logique

```
#!/bin/bash
# backup.sh – exécuter depuis /opt/ritoto-express/
DATE=$(date +%Y%m%d-%H%M)
BACKUP_DIR="/opt/backups/ritoto"
mkdir -p "$BACKUP_DIR"

# 1. Dump PostgreSQL
docker compose -f docker-compose.prod.yml exec -T postgres \
  pg_dump -U ritoto ritoto > "$BACKUP_DIR/db-$DATE.sql"

# 2. Archive media
tar -czf "$BACKUP_DIR/media-$DATE.tar.gz" -C /opt/ritoto-express media/

# 3. Archive complète
tar -czf "$BACKUP_DIR/ritoto-backup-$DATE.tar.gz" \
  "$BACKUP_DIR/db-$DATE.sql" \
  "$BACKUP_DIR/media-$DATE.tar.gz"

# 4. Supprimer les fichiers intermédiaires
rm "$BACKUP_DIR/db-$DATE.sql" "$BACKUP_DIR/media-$DATE.tar.gz"

# 5. Garder seulement les 14 derniers backups
ls -t "$BACKUP_DIR"/ritoto-backup-*.tar.gz | tail -n +15 | xargs -r rm

echo "Backup $DATE terminé."
```

Activation du cron

```
# Backup quotidien à 2h du matin
0 2 * * * /opt/ritoto-express/docker/scripts/backup.sh >> /var/log/ritoto-backup.log 2>&1

# Renouvellement SSL à 3h du matin
0 3 * * * certbot renew --quiet && docker compose -f /opt/ritoto-express/docker-compose.prod
```


PHASE 8

Mise en ligne & checklist de vérification

- ☐ <https://ritoto-campus.com> → Frontend React s'affiche correctement
- ☐ <https://api.ritoto-campus.com/api/> → API répond en JSON (pas d'erreur 500)
- ☐ <https://api.ritoto-campus.com/admin/> → Interface admin Django accessible
- ☐ Connexion / inscription d'un étudiant de test fonctionne
- ☐ Ajouter un produit au panier et passer une commande
- ☐ Vérifier que le chef de secteur reçoit la notification en temps réel
- ☐ Valider la commande → l'étudiant voit le statut changer instantanément
- ☐ Webhook Senfenico configuré dans le dashboard Senfenico (URL prod)
- ☐ Paiement test avec Senfenico (clé production)
- ☐ Certificat SSL valide (cadenas vert dans le navigateur)
- ☐ Backup automatique activé et testé (vérifier `/var/log/ritoto-backup.log`)
- ☐ Tester la restauration depuis un backup sur un environnement de test



Webhook Senfenico : Configurer l'URL de callback dans le dashboard Senfenico vers <https://api.ritoto-campus.com/api/commandes/webhook/> . C'est ce webhook qui déclenche l'envoi de la notification WebSocket à l'étudiant après confirmation du paiement.

PHASE 9

Opérations courantes

Déployer une mise à jour backend

```
cd /opt/ritoto-express
git pull
docker compose -f docker-compose.prod.yml build backend daphne
docker compose -f docker-compose.prod.yml up -d --no-deps backend daphne
docker compose exec backend python manage.py migrate
```

Mettre à jour le frontend uniquement

```
cd /opt/ritoto-express/frontend
npm run build
docker compose -f docker-compose.prod.yml restart frontend nginx
```

Consulter les logs en temps réel

```
# Logs API REST (erreurs Python, requêtes)
docker compose -f docker-compose.prod.yml logs -f backend

# Logs WebSocket (connexions, déconnexions)
docker compose -f docker-compose.prod.yml logs -f daphne

# Logs Nginx (accès, erreurs SSL)
docker compose -f docker-compose.prod.yml logs -f nginx

# Logs PostgreSQL
docker compose -f docker-compose.prod.yml logs -f postgres
```

Accéder au shell Django (debug/maintenance)

```
docker compose -f docker-compose.prod.yml exec backend python manage.py shell
```

Restaurer depuis un backup

```
# 1. Arrêter les conteneurs
docker compose -f docker-compose.prod.yml down

# 2. Restaurer la base de données
docker compose -f docker-compose.prod.yml up -d postgres
docker compose exec -T postgres psql -U ritoto ritoto < /opt/backups/ritoto/db-YYYYMMDD.sql

# 3. Restaurer les media
tar -xzf /opt/backups/ritoto/media-YYYYMMDD.tar.gz -C /opt/ritoto-express/

# 4. Redémarrer la stack complète
docker compose -f docker-compose.prod.yml up -d
```

Annexe — Points d'attention & sécurité



Sécurité critique : Ne jamais committer le fichier `.env` sur Git. Vérifier que `.gitignore` exclut bien `.env`, `.env.production`, et tous les fichiers `*.env`. La `DJANGO_SECRET_KEY` doit être différente en dev et en prod.

Point	Risque	Solution
<code>DEBUG=True</code> en prod	● Expose le code source et les variables d'env	Toujours <code>False</code> en production
WebSocket non authentifié	● N'importe qui peut se connecter	<code>AuthMiddlewareStack</code> + vérification <code>user.is_authenticated</code> dans le consumer
PostgreSQL exposé	● Accès direct à la base de données	PostgreSQL sur réseau Docker interne uniquement, pas de port exposé vers l'extérieur
Connexions WS longues	● Saturation des ressources serveur	Nginx <code>proxy_read_timeout 86400</code> , Daphne gère la déconnexion proprement
Clé Senfenico de test en prod	● Paiements non réels	Utiliser impérativement les clés production du dashboard Senfenico
Pas de backup	● Perte totale des données si disque corrompu	Script <code>backup.sh</code> + cron quotidien + stockage distant
Certificat SSL expiré	● Site inaccessible, erreur navigateur	Certbot renew en cron + monitoring Uptime Robot

Annexe — Monitoring (optionnel)



Uptime Robot (gratuit)

Surveillance uptime toutes les 5 minutes. Alerte email/SMS si le site tombe. Surveiller à minima

`https://ritoto-campus.com` et `https://api.ritoto-campus.com/api/`.



Sentry (gratuit jusqu'à un seuil)

Tracking des erreurs Python (backend) et JavaScript (frontend). Chaque exception en production est capturée avec la stack trace complète.

